



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Diseño y Análisis de Algoritmos - IIC2283 - 1^{er} semestre - 2026

Tarea 1

Dividir y conquistar y Programación Dinámica

Publicación : Jueves 09 de abril
Github Classroom : <https://classroom.github.com/a/SYsbxbvV>
Entrega : Jueves 23 de abril

Indicaciones

- La tarea es estrictamente individual.
- La solución debe ser entregada en los archivos `t1p1.py` y `t1p2.py` del repositorio privado asignado mediante GitHub Classroom para esta tarea. Se revisará el último *commit* subido antes de la entrega al repositorio. Se usará Python 3.12.X para la revisión.
- El *input* para el programa debe ser obtenido desde *standard input*. El *output* debe ser entregado mediante *standard output*.
- La corrección se realizará mediante *tests* automatizados acordes al formato de *input* y *output* especificado. Cada *test* tendrá un *timeout* según lo que se especifica como tiempo esperado.
- Un *test* se considerará **reprobado** en caso de que 1) dado el *input* el *output* sea incorrecto, 2) exista un error de *runtime* durante su ejecución, o 3) el *timeout* se cumpla durante su ejecución. En otro caso, el *test* se considerará **aprobado**.
- No se permite el uso de librerías externas a la librería estándar de Python *a priori*. Consultar en las [issues del repositorio oficial del curso](#) en caso de requerir una.
- A 5 días de la fecha final del plazo, se publicará un test de gran tamaño para cada parte, siendo el único test que publicaremos. La habilidad de generar tests es parte de la evaluación.
- Para esta tarea aplica la política de atrasos descrita en el programa del curso.

1. Brazo Robótico (50 %)

Para un curso de robótica, los ayudantes han construido un nuevo brazo robótico inteligente para ordenar y armar los kits de hardware que se entregarán a los alumnos. Para optimizar el consumo de batería durante este proceso, programaron al robot con una versión adaptada del algoritmo Merge sort.

La función principal que ordena los componentes del arreglo A con índices en el rango $[l, r]$ opera de la siguiente manera:

1. Si el segmento $[l, r]$ ya está ordenado en orden no descendente (es decir, para cualquier i tal que $l \leq i < r$, se cumple que $A[i] \leq A[i + 1]$), entonces la llamada a la función termina.
2. Sea $mid = \lfloor \frac{l+r}{2} \rfloor$.
3. Se llama a $mergesort(A, l, mid)$.
4. Se llama a $mergesort(A, mid + 1, r)$.
5. Se fusionan los segmentos $[l, mid]$ y $[mid + 1, r]$, haciendo que el segmento $[l, r]$ quede ordenado en orden no descendente. El algoritmo de fusión no realiza ninguna otra llamada a funciones.

El arreglo en este sistema está indexado desde 0, por lo que para ordenar toda la fila de componentes, el sistema inicializa el proceso llamando a $mergesort(A, 0, n - 1)$.

Los ayudantes están realizando pruebas de estrés a la batería del brazo robótico. Saben que cada vez que se hace una llamada a la función *mergesort*, el robot consume exactamente 1 unidad de energía. Para calibrar los sistemas, necesitan armar un lote de prueba con n componentes, los cuales deben formar una permutación de tamaño n (es decir, el arreglo A tiene n elementos y contiene cada entero del 1 al n exactamente una vez). Además, el lote debe estar ordenado de tal forma que el robot consuma **exactamente** k unidades de energía al ordenarlo.

¡Ayuda a los ayudantes a encontrar la disposición inicial de los componentes que cumpla con estos requisitos!

Input

La única línea de la entrada consiste de dos enteros: n y k ($1 \leq n \leq 10^5$, $1 \leq k \leq 2 \cdot 10^5$), correspondientes a la cantidad de componentes en la fila y la cantidad de unidades de energía requeridas (llamadas a la función), respectivamente.

Output

Si no existe una permutación de tamaño n que consuma exactamente k unidades de energía al ser ordenada, imprime -1. En caso contrario, imprime n números enteros $A[0], A[1], \dots, A[n - 1]$ separados por espacios, correspondientes a los elementos de una permutación que cumpla las condiciones requeridas. Si existen múltiples respuestas válidas, imprime cualquiera de ellas.

Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 2 segundos** para cualquier instancia de input según las restricciones dadas.

Complejidad esperada

Se espera que la solución posea una complejidad de $O(n)$.

Adicional

Soluciones que no utilicen el paradigma dividir y conquistar no serán consideradas y serán evaluadas con la nota mínima.

Ejemplos

Input de ejemplo 1
3 3
Output de ejemplo 1
1 3 2

Input de ejemplo 2
4 1
Output de ejemplo 2
1 2 3 4

Input de ejemplo 3
3 2
Output de ejemplo 3
-1

2. Parkour (50 %)

Michael Scott está diseñando la pista de obstáculos definitiva para su próximo entrenamiento de parkour. El terreno disponible se puede representar como una cuadrícula unidimensional de N celdas, donde cada celda está inicialmente compuesta por suelo firme y seguro, o por zonas peligrosas llenas de barro y espinas.



Figura 1: El equipo de parkour evaluando el terreno para su circuito

Michael puede conseguir plataformas de salto de cualquier longitud. Una plataforma de longitud k ocupa exactamente k celdas consecutivas de suelo seguro. No es posible colocar más de una plataforma en la misma celda. Para que el circuito luzca estético y el entrenamiento sea progresivo, Michael decidió que todas las plataformas que instale deben tener longitudes distintas y estar dispuestas en orden estrictamente creciente de izquierda a derecha.

Por cada plataforma que Michael logra instalar, un patrocinador le paga una ganancia G . Sin embargo, Michael no está satisfecho con el terreno inicial, por lo que está dispuesto a pavimentar algunas celdas peligrosas para convertirlas en suelo seguro y así poder colocar más plataformas. Pavimentar la celda i -ésima tiene un costo variable T_i , ya que no todas las celdas requieren la misma cantidad de trabajo.

Si Michael elige sabiamente qué celdas pavimentar y dónde colocar las plataformas, ¿cuál es la máxima ganancia neta que puede obtener?

Input

La primera línea contiene dos enteros N y G ($1 \leq N \leq 10^5$, $1 \leq G \leq 10^9$), que indican respectivamente la cantidad de celdas del terreno y la ganancia por colocar una plataforma.

La segunda línea contiene N enteros $T_1, T_2, \dots, T_N$ ($0 \leq T_i \leq 10^9$), tal que $T_i = 0$ si la celda i -ésima ya es segura, o $T_i \geq 1$ si es peligrosa y su costo de pavimentación es T_i .

Output

La salida debe contener un único entero correspondiente a la máxima ganancia neta que Michael puede alcanzar.

Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 3 segundos** para cualquier instancia de input según las restricciones dadas.

Complejidad esperada

Se espera que la solución posea una complejidad de $O(N\sqrt{N})$, lo cual es suficiente bajo los límites del problema.

Ejemplos

Input de ejemplo 1
5 10 0 2 0 5 0
Output de ejemplo 1
18

Input de ejemplo 2
10 2 0 0 0 0 8 0 0 7 0 0
Output de ejemplo 2
4

Input de ejemplo 3
4 1 2 2 2 2
Output de ejemplo 3
0

Explicación de los casos

En el primer caso de prueba podemos instalar una plataforma de longitud 1 en la primera celda y una segunda plataforma de longitud 2 en la segunda y tercera celda. Como las celdas tienen longitudes distintas y están dispuestas en orden creciente de izquierda a derecha esta instalación es válida.

Como se instalaron 2 plataformas, Michael consigue una ganancia de 20, pero para ocupar la segunda celda, primero se debió pavimentar con un coste de 2, por lo que la ganancia total es de 18 y esta es la respuesta del caso de prueba.

En el tercer caso de prueba, instalar cualquier cantidad de plataformas genera más costos que ganancias, por lo que la mayor ganancia posible es de 0 (no instalar ninguna plataforma).

Política de integridad académica

Los/as estudiantes de la Escuela de Ingeniería de la Pontificia Universidad Católica de Chile deben mantener un comportamiento acorde a la Declaración de Principios de la Universidad. En particular, se espera que mantengan altos estándares de honestidad académica. Cualquier acto deshonesto o fraude académico está prohibido; los/as estudiantes que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada estudiante conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Docencia de la Escuela de Ingeniería.

Específicamente, para los cursos del Departamento de Ciencia de la Computación, rige obligatoriamente la siguiente política de integridad académica.

Todo trabajo presentado por un/a estudiante para los efectos de la evaluación de un curso debe ser hecho por el/la estudiante. El/la estudiante es responsable de originar el material entregado y de conocer íntegramente su contenido. Asimismo, deberá respetar estrictamente las condiciones definidas por el curso para cada trabajo (por ejemplo, si el trabajo es individual, grupal, y si se permite o no el uso de material digital o de asistentes de inteligencia artificial). Por “trabajo” se entiende en general las interrogaciones escritas, las tareas de programación u otras, los trabajos de laboratorio, los proyectos, el examen, entre otros.

Si un/a estudiante incurre en una falta a la integridad académica, como, por ejemplo, entregando trabajo que no es propio, no comprendiendo íntegramente el trabajo entregado, o no cumpliendo con las condiciones establecidas por el curso, obtendrá nota final 1.1 en el curso y se solicitará a la Dirección de Pregrado de la Escuela de Ingeniería que no le permita retirar el curso de la carga académica semestral. En todos los casos, se informará a la Comisión de Integridad Académica de la Escuela de Ingeniería para que tome sanciones adicionales si lo estima pertinente.

En caso de utilización de fuentes digitales como Wikipedia o el uso de asistentes inteligentes como ChatGPT o Copilot, cuando se permita su uso, debe declararse de forma explícita el material o herramienta usada. En caso de uso de asistentes de inteligencia artificial, el profesor/a del curso puede solicitar que se entreguen respaldos sobre su utilización (por ejemplo, historial o logs).

Lo anterior se entiende como complemento al Reglamento del Estudiante de la Pontificia Universidad Católica de Chile (<https://registrosacademicos.uc.cl/reglamentos/estudiantiles/>). Por ello, es posible pedir a la Universidad la aplicación de sanciones adicionales especificadas en dicho reglamento.

Compromiso del Código de Honor

Este curso suscribe el Código de Honor establecido por la Universidad, el que es vinculante. Todo trabajo evaluado en este curso debe ser propio. En caso que exista colaboración permitida con otros/as estudiantes, el trabajo deberá referenciar y atribuir correctamente dicha contribución a quien corresponda. Como estudiante es un deber conocer el Código de Honor (<https://www.uc.cl/codigo-de-honor/>).